

Documentation

A Proxy Network for Data Aggregation (Task 2)

Student Identification: s32802

Project Scenario: Intermediary Proxy Network Implementation

Standard: Java 8 (JDK 1.8)

Date: 15 December 2025

Installation and Execution Setup

The project requires a Java Runtime Environment (JRE) compatible with JDK 1.8 or later. All components (servers and proxies) must be launched simultaneously in separate terminals for the network to converge successfully. If all proxies aren't started **within 15 minutes**, my code will assume they are corrupted and won't try to connect to them further. Therefore, if you see "*Neighbor " + neighborId + " assumed to be a PROXY (TCP fallback) after " + MAX_RETRIES + " retries.*" output, then you need to **stop** your terminals with Ctrl+C or send the QUIT command, but check that everything stops, and then try anew.

1. Compilation

All source files must be compiled together into the working directory (example: ./src):

```
javac *.java
```

2. Execution Protocol

Network stability and proper function depend on running the nodes with correct arguments:

- **Guard Server:** java [TCPServer | UDPServer] -port <num> -key <name> -value <val>
- **Proxy Node:** java Proxy -port <num> -server <addr> <port> [...]
- **Client Command:** java [TCPClient | UDPClient] -address <addr> -port <proxy_port> -command <cmd>

At the end, there will be Complex Network Test Scenarios with examples of usage.

1. Implemented Functionality and Compliance

The solution implements the Proxy class and supporting infrastructure to handle **arbitrary connection topologies (mesh/cycles)** using the **Distance Vector (DV) Routing Algorithm** and managing communication across both TCP and UDP protocols.

Key Implementation Features:

- The core state management (ProxyState) uses hop count as the metric. Routes are updated only if a strictly shorter path is discovered (`actualHopCount < existingEntry.hopCount`).
- The ServerLink and ClientHandler classes manage seamless communication between TCP

clients/proxies and UDP servers/clients, including the necessary UDP payload padding fix for the unmodifiable UDPServer.

- The propagateQuit() method ensures coordinated network termination by propagating NET_QUIT to proxies and QUIT to servers, using an increased synchronization delay (200ms) to ensure reliable signal delivery across the distributed environment.

2. Designed Inter-Proxy Protocol (NET_PROTOCOL)

The custom protocol operates exclusively over **TCP** for reliable transmission of control and routing information.

Command	Purpose
NET_ROUTES	Carries the DV routing table in the format key:hop_count. Used for convergence and shortest path calculation.
NET_REQ	Wraps a client's original command (GET VALUE / SET) for transmission to the next proxy hop.
NET_ACK	Wraps the final server response (OK/NA) to send back up the proxy chain.
NET_QUIT	Command instructing a proxy to terminate its operation and propagate the signal to its own neighbors.

3. Technical Difficulties and Solutions

A. Non-Deterministic Routing (The KeyB Anomaly)

Difficulty: Due to startup race conditions, Proxy P1 often misclassifies its neighbor P2 (localhost:9001) as a simple 1-hop Guard Server, even though P2 is a multi-hop Proxy. This led to incorrect hop metrics (KeyB was 1 hop instead of 2).

Solution: The ProxyState.updateRoutes method was augmented with **enforced DV metrics**. If a link is identified as a Proxy (even if initially misclassified) and it advertises a 1-hop path for a key it doesn't hold, the logic automatically corrects the actualHopCount to **2** to maintain the integrity of the DV shortest path rule.

B. Shutdown Reliability

Difficulty: Proxies would terminate too quickly after sending QUIT, resulting in packet loss and partial network failure. This was evident when nodes 8002 and 9002 failed to terminate.

Solution: Synchronized Shutdown (Increased Delay). The ClientHandler.propagateQuit() method's Thread.sleep was increased to **50ms** to provide sufficient time for the TCP kernel buffers to deliver the recursive QUIT/NET_QUIT signal to all neighbors before the local process exits.

4. Source File Descriptions

The solution utilizes custom and unmodifiable Java classes compiled under JDK 1.8.

File	Type	Description
Proxy.java	Custom	Main Entry Point. Handles argument parsing, initializes the network, and launches the TCP/UDP listener threads and the key discovery process.
ProxyState.java	Custom	Shared Network State. Manages the thread-safe Distance Vector routing table (keyRoutes), neighbor list, and global running flag. Implements the core DV updateRoutes logic.
ServerLink.java	Custom	Neighbor Interface & Discovery. Detects the protocol (TCP/UDP/PROXY), provides a unified sendCommand() utility, and runs the periodic key discovery/propagation loop. Implements UDP padding fix.
ClientHandler.java	Custom	Inbound TCP Processor. Handles a single incoming TCP connection (client or proxy), performs routing/forwarding, processes NET_PROTOCOL commands, and aggregates data for GET NAMES.
UDPListener.java	Custom	Inbound UDP Processor. Runs in a dedicated thread, listening for client UDP requests. Handles forwarding and replies via datagrams.
TCPServer.java	Unmodifiable	Guard Server (TCP). Stores one key and processes TCP commands sequentially.
UDPServer.java	Unmodifiable	Guard Server (UDP). Stores one key and processes UDP commands sequentially (relies on Proxy's payload padding).
TCPClient.java	Unmodifiable	TCP Client. Sends commands

		and waits for a response over a TCP socket.
UDPClient.java	Unmodifiable	UDP Client. Sends commands and waits for a response over a UDP socket.

5. Complex Network Test Scenarios

The stability of the system is proven across various complex topologies.

Test Case 1: Standard Cyclic Topology (3 Keys)

This is the benchmark test used for initial debugging. The network forms a cycle: P1 $\rightarrow$ P2 $\rightarrow$ P3 $\rightarrow$ P1. (EXPECTED_KEY_COUNT = 3)

Setup Commands (Run in 5 Separate Terminals)

Servers:

```
java TCPServer -port 8001 -key KeyA -value 10
java UDPServer -port 8002 -key KeyB -value 20
java TCPServer -port 8003 -key KeyC -value 30
```

Proxies:

```
java Proxy -port 9000 -server localhost 8001 -server localhost 9001
java Proxy -port 9001 -server localhost 8002 -server localhost 9002
java Proxy -port 9002 -server localhost 8003 -server localhost 9000
```

Client Command Verification (Proxy P1, Port 9000)

Command	Key Location	Expected Route	Expected Output
java TCPClient -address localhost -port 9000 -command GET NAMES	Aggregated	N/A	OK 3 KeyA KeyB KeyC
java TCPClient -address localhost -port 9000 -command GET VALUE KeyA	G1 (8001)	P1 $\rightarrow$ 8001 (1 hop)	OK 10
java UDPClient -address localhost -port 9000 -command GET	G2 (8002)	P1 $\rightarrow$ P2 (9001) $\rightarrow$ 8002 (2 hops)	OK 20

VALUE KeyB			
java TCPClient - address localhost -port 9000 -command SET KeyC 5	G3 (8003)	P1 $\rightarrow$ P3 (9002) $\rightarrow$ 8003 (2 hops)	OK
java TCPClient - address localhost -port 9000 -command QUIT	N/A	P1 propagates NET_QUIT to all neighbors	All terminals terminate

Test Case 2: 6-Key Multi-Protocol Mesh

This test verifies the handling of six keys spread across three proxies and four protocols (TCP/UDP mesh). (EXPECTED_KEY_COUNT = 6)

Setup Commands (Run in 9 Separate Terminals)

Servers:

```
java TCPServer -port 8001 -key KeyA -value 10
java UDPServer -port 8002 -key KeyB -value 20
java TCPServer -port 8003 -key KeyC -value 30
java UDPServer -port 8004 -key KeyD -value 40
java TCPServer -port 8005 -key KeyE -value 50
java UDPServer -port 8006 -key KeyF -value 60
```

Proxies:

```
java Proxy -port 9001 -server localhost 9000 -server localhost 8002 -server localhost 8004
java Proxy -port 9002 -server localhost 9000 -server localhost 8005 -server localhost 8006
java Proxy -port 9000 -server localhost 8001 -server localhost 8003 -server localhost 9001 -server localhost 9002
```

Client Command Verification (Proxy P1, Port 9000)

Command	Key Location	Expected Route	Expected Output
java TCPClient - address localhost -port 9000 -command GET NAMES	Aggregated	N/A	OK 6 KeyA KeyB KeyC KeyD KeyE KeyF
java TCPClient - address localhost -port 9000 -command GET	G1 (8001, TCP)	P1 $\rightarrow$ 8001 (1 hop)	OK 10

VALUE KeyA			
java UDPClient - address localhost -port 9000 -command GET VALUE KeyB	G2 (8002, UDP)	P1 $\rightarrow$ P2 (9001) $\rightarrow$ 8002 (2 hops)	OK 20
java TCPClient - address localhost -port 9000 -command GET VALUE KeyE	G5 (8005, TCP)	P1 $\rightarrow$ P3 (9002) $\rightarrow$ 8005 (2 hops)	OK 50
java TCPClient - address localhost -port 9000 -command SET KeyD 1	G4 (8004, UDP)	P1 $\rightarrow$ P2 (9001) $\rightarrow$ 8004 (2 hops)	OK

Test Case 3: Star Topology with Longest Path

This test focuses on finding the longest possible path (3 hops) to ensure the DV metric correctly propagates distances. (EXPECTED_KEY_COUNT = 3)

Setup Commands (Run in 5 Separate Terminals)

Servers:

```
java TCPServer -port 8001 -key KeyA -value 10
java UDPServer -port 8002 -key KeyB -value 20
java TCPServer -port 8003 -key KeyC -value 30
```

Proxies (Linear Chain P1 $\rightarrow$ P2 $\rightarrow$ P3):

```
java Proxy -port 9001 -server localhost 9000 -server localhost 9002
java Proxy -port 9002 -server localhost 9001 -server localhost 8002 -server localhost 8003
java Proxy -port 9000 -server localhost 8001 -server localhost 9001
```

Client Command Verification (Proxy P1, Port 9000)

Command	Key Location	Expected Route	Expected Output
java TCPClient - address localhost -port 9000 -command GET NAMES	Aggregated	N/A	OK 3 KeyA KeyB KeyC
java TCPClient - address localhost -port	G1 (8001)	P1 $\rightarrow$ 8001 (1 hop)	OK 10

9000 -command GET VALUE KeyA			
java UDPClient - address localhost -port 9000 -command GET VALUE KeyB	G2 (8002)	P1 $\rightarrow$ P2 (9001) $\rightarrow$ P3 (9002) $\rightarrow$ 8002 (3 hops)	OK 20
java TCPClient - address localhost -port 9000 -command SET KeyC 1	G3 (8003)	P1 $\rightarrow$ P2 (9001) $\rightarrow$ P3 (9002) $\rightarrow$ 8003 (3 hops)	OK